

Since you've been working with **Playwright + TypeScript**, here's the complete **step-by-step Allure Report setup** using the latest Playwright versions.

Step 1: Install Allure packages

From your Playwright project root, install:

 Bash 

```
npm install -D allure-playwright allure-commandline
```

Verify:

 Bash 

```
npm list allure-playwright
npm list allure-commandline
```

Expected:

```
allure-playwright@3.x.x
allure-commandline@2.x.x
```



Step 2: Configure playwright.config.ts

Add the Allure reporter.

 TypeScript 

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  reporter: [
    ['html'],
    ['list'],
    ['allure-playwright']
  ],

});
```

Or with output folder:

❏ TypeScript



```
reporter: [  
  ['html'],  
  ['allure-playwright', {  
    outputFolder: 'allure-results'  
  }]  
]  
]
```

Your project should look like:

```
project  
├──  
├─ playwright.config.ts  
├─ package.json  
├─ tests  
│   └─ login.spec.ts  
├─  
├─ allure-results  
└─ allure-report
```



Step 3: Execute tests

Run

❏ Bash



```
npx playwright test
```

After execution,

```
allure-results/
```



should be created automatically.

Example

```
allure-results  
  
4a5d-result.json  
6be7-result.json  
attachments/
```



If this folder is not created, Allure is not configured correctly.

Step 4: Generate report

Run

 Bash



```
npx allure generate allure-results --clean
```

Output

```
allure-report/
```



is created.

Step 5: Open report

Run

 Bash



```
npx allure open allure-report
```

Browser opens automatically.

You'll see

Overview



Suites

Graphs

Timeline

Behaviors

Packages

Categories

History

Shortcut

Instead of generate + open

Run

❏ Bash



```
npx allure serve allure-results
```

This

- generates report
- starts server
- opens browser

in one command.

Step 6: Add scripts in package.json

❏ JSON



```
{
  "scripts": {
    "test": "playwright test",

    "allure:generate": "allure generate allure-results --clean",

    "allure:open": "allure open allure-report",

    "allure:serve": "allure serve allure-results"
  }
}
```

Now use

```
npm test
```



```
npm run allure:generate
```

```
npm run allure:open
```

or

```
npm run allure:serve
```



Step 7: Attach screenshots automatically

In `playwright.config.ts`

❏ TypeScript



```
use: {  
  screenshot: "only-on-failure",  
  trace: "retain-on-failure",  
  video: "retain-on-failure"  
}
```

These files are available in Playwright artifacts. To explicitly add them to Allure, you can attach them in your test hooks (shown below).

Step 8: Add Allure annotations

Inside a test

❏ TypeScript



```
import { test } from '@playwright/test';  
import * as allure from 'allure-js-commons';  
  
test('Login Test', async ({ page }) => {  
  
  await allure.owner("Bala");  
  
  await allure.severity("critical");  
  
  await allure.epic("Authentication");  
  
  await allure.feature("Login");  
  
  await allure.story("Valid Login");  
  
  await page.goto("https://example.com");  
});
```

Report displays

- Owner
 - Severity
 - Epic
 - Feature
 - Story
-

Step 9: Add steps

❏ TypeScript



```
await allure.step("Open Login Page", async () => {

    await page.goto("https://example.com");

});

await allure.step("Enter Username", async () => {

    // code

});

await allure.step("Click Login", async () => {

    // code

});
```

Report

Login Test



Open Login Page

Enter Username

Click Login

Step 10: Attach screenshots

❏ TypeScript



```
import * as allure from 'allure-js-commons';

const screenshot = await page.screenshot();

await allure.attachment(
    "Home Page",
    screenshot,
    {
        contentType: "image/png"
    }
);
```

```
}  
);
```

Screenshot appears directly in the report.

Step 11: Attach text

❏ TypeScript



```
await allure.attachment(  
  "Response Body",  
  JSON.stringify(responseBody, null, 2),  
  {  
    contentType: "application/json"  
  }  
);
```

Useful for API testing.

Step 12: Attach request/response

❏ TypeScript



```
await allure.attachment(  
  "Request",  
  JSON.stringify(requestBody, null, 2),  
  {  
    contentType: "application/json"  
  }  
);  
  
await allure.attachment(  
  "Response",  
  JSON.stringify(responseBody, null, 2),  
  {  
    contentType: "application/json"  
  }  
);
```

This is especially helpful when using Playwright's `APIRequestContext`.

Step 13: Folder structure after execution

```
project

tests

playwright.config.ts

package.json

allure-results
  *.json
  *.txt
  attachments/

allure-report
  index.html
  data/
```



Step 14: Typical execution flow

```
Run Tests
  |
  ▼
Playwright executes tests
  |
  ▼
allure-results created
  |
  ▼
Generate Report
  |
  ▼
allure-report created
  |
  ▼
Open Report
  |
  ▼
Interactive HTML Dashboard
```



Common errors

1. `allure-results` folder not created

Check:

❏ TypeScript

```
reporter: [  
  ['allure-playwright']  
]
```



2. allure: command not found

Install:

❏ Bash

```
npm install -D allure-commandline
```



3. Empty report

Generate after running tests:

❏ Bash

```
npx playwright test
```

```
npx allure generate allure-results --clean
```



4. Old results still visible

Clean results:

❏ Bash

```
rm -rf allure-results allure-report
```



On Windows PowerShell:

❏ PowerShell

```
Remove-Item -Recurse -Force allure-results, allure-report
```



Then rerun tests.

Recommended configuration for your Playwright project

TS TypeScript



```
reporter: [
  ['list'],
  ['html', { open: 'never' }],
  ['allure-playwright', { outputFolder: 'allure-results' }]
],

use: {
  screenshot: 'only-on-failure',
  trace: 'retain-on-failure',
  video: 'retain-on-failure'
}
```

This setup lets you use Playwright's built-in HTML report during development while generating rich Allure reports with attachments for test execution and CI pipelines.